

Programming for Robotics: Project Report

Project Title: Autonomous Irrigation Unit (A.I.U)

GitHub Repository Link: <https://github.com/DobTechTD/3003ICT-Project-AIU-Group-14>

Video Link: <https://youtu.be/YA3cA9fydmg>

Group Members:

- Thomas Dobson – s5434264

Specialisation Option Chosen:

☒ Embedded System (Wokwi)

1. Project Overview

The Autonomous Irrigation Unit (A.I.U) is an Arduino Uno embedded controller simulated in Wokwi. The system monitors soil moisture, temperature, light level, and tank water level to manage irrigation and protect the pump from unsafe operation.

The system uses sensor fusion, a five-state FSM, adaptive polling, day/night heat-risk adjustment, low-water lockout, and failsafe protection. Irrigation only occurs when soil is dry enough, temperature indicates increased irrigation demand, water level is safe, and sensor readings are valid. The intended environment is a small-scale agricultural or greenhouse system where irrigation should respond automatically to changing conditions.

2. Technical Requirements Compliance

<i>Requirement</i>	<i>Implementation</i>
Wokwi embedded system	Arduino Uno simulation with environmental component inputs and output devices.
Inputs	DHT22 temperature sensor, photoresistor for day/night, soil moisture slider potentiometer, water level potentiometer, and the pushbutton.
Outputs	Relay/pump, RGB LED, buzzer, LCD 2004 I2C visual display, and the Serial Monitor for wokwi.
Safety	Low-water lockout, failsafe, sensor validity check, timeout and hysteresis thresholds.
Advanced behaviour	Sensor fusion, adaptive polling rates, adaptive day/night heat-risk thresholds.
Code quality	Modular Arduino tabs for ease-of-use configuration, sensors, FSM, outputs, display, and button input
Behaviour model	Five-state FSM: IDLE, MONITORING, IRRIGATING, WARNING, FAILSAFE

3. System Architecture

Sensors → Processing → Decision → Act / Outputs

DHT22 temperature sensor, soil moisture potentiometer, water level potentiometer, photoresistor/light sensor, and pushbutton → Arduino Uno processing: read sensor values, map analogue inputs, check sensor validity, apply hysteresis, apply day/night context, and adjust adaptive polling → FSM decision logic: IDLE, MONITORING, IRRIGATING, WARNING, and FAILSAFE states using sensor fusion, timeout logic, low-water lockout, and failsafe conditions → relay/pump, RGB LED state indicator, buzzer warning output, LCD 2004 I2C display, and Serial Monitor.

4. Behaviour Model

<u>State</u>	<u>Purpose</u>	<u>Main outputs</u>
IDLE	Safe startup/standby	Pump off, buzzer off, blue LED
MONITORING	Normal sensor checking	Pump off, green LED, LCD updates
IRRIGATING	Active irrigation	Pump on, cyan LED
WARNING	Non-critical unsafe condition	Pump off, yellow LED, warning buzzer
FAILSAFE	Critical safety lockout	Pump off, red LED, fast buzzer

<u>Transition</u>	<u>Trigger</u>
IDLE → MONITORING	Button pressed
MONITORING → IRRIGATING	Dry soil + high temperature + safe water
MONITORING//IRRIGATING → WARNING	Low water, severe heat/dryness, or timeout
WARNING → MONITORING	Warning condition clears
ANY ACTIVE STATE → FAILSAFE	Critical water level or invalid sensor
FAILSAFE → IDLE	Water recovered and button pressed

An FSM is used because the system has clear operating modes with different outputs, safety rules, and transition conditions. As well as the system only needs to be active in one state at one time.

5. Perception / Sensor Processing

<u>Sensor</u>	<u>Data read/ Processing</u>	<u>Decision affect</u>
DHT22	Temperature in °C	Determines high temperature and heat warning flags
Soil moisture potentiometer	Soil moisture percentage	Determines dry and severely dry conditions
Water level potentiometer	Tank water percentage	Determines low-water warning and critical failsafe
Photoresistor	Light percentage	Determines day/night context
Pushbutton	Press event/ Debounced input	Starts system, acknowledges warnings, resets failsafe

Sensor fusion is used because irrigation does not depend on one input alone. The system only enters IRRIGATING when soil is dry, temperature is high, water is safe, sensor readings are valid, and the irrigation cooldown has finished. The light sensor provides day/night context by changing the heat-risk thresholds. Hysteresis prevents rapid switching near thresholds, such as soil becoming dry below 40% but not clearing until it rises above 50%.

6. Safety and Robustness

<i>Feature</i>	<i>Description</i>
Low-water lockout	Pump is disabled when water is low-threshold.
Critical-water failsafe	Critical water level or invalid sensor readings force FAILSAFE.
Sensor validity checking	millis() allows continuous sensing, output updates, and timeout supervision.
Irrigation timeout	Pump cannot run indefinitely during IRRIGATING.
Hysteresis	Prevents flickering near thresholds, such as soil dry/clear or water low/clear.
Manual reset/acknowledge	The system only exits failsafe after safe recovery and button input.

The safety design prioritises protecting the pump and preventing unsafe irrigation. The system always checks water availability and sensor validity before allowing irrigation, and the pump is forced off in all warning and failsafe conditions.

7. Code Structure

<i>File</i>	<i>Purpose</i>
sketch.ino	Contains setup() and loop(), giving the main execution flow
Config.h	Stores libraries, pin definitions, thresholds, FSM enum, timing variables, sensor variables, and global flags
Sensors.ino	Reads sensors, maps raw data to percentages, updates hysteresis flags, and controls adaptive polling intervals
FSM.ino	Contains state transition logic, cooldown logic, and state name helper
Outputs.ino	Maps FSM states to relay, buzzer, and RGB LED outputs
Display.ino	Updates the LCD and Serial Monitor
Button.ino	Handles debounced pushbutton input
DHT.h, Wire.h, LiquidCrystal_I2C.h, Arduino core functions	Reads temperature data from the DHT22 sensor for heat-risk detection. Enables I2C communication between the Arduino Uno and LCD display. Controls the 20x4 I2C LCD used to display live readings, state, and warnings. Uses millis(), analogRead(), digitalWrite(), tone(), and pinMode() for timing, sensing, and output control.

Main loop flow: button input → adaptive sensor read → hysteresis update → FSM update → output control → LCD/Serial update.

Key libraries used were DHT.h for the DHT22 temperature sensor, Wire.h for I2C communication, and LiquidCrystal_I2C.h for the LCD. Arduino core functions such as millis(), analogRead(), digitalWrite(), tone(), and pinMode() support timing, sensing, and output control. For the basic functionality.

8. Key Code Snippets

<u>Key code snippet</u>	<u>Purpose / explanation</u>
cpp enum SystemState { IDLE, MONITORING, IRRIGATING, WARNING, FAILSAFE };	This enum defines the five possible operating states of the controller.
cpp unsigned long getSensorInterval() { switch (currentState) { case IDLE: return 2000; case MONITORING: return 1000; case IRRIGATING: return 500; case WARNING: return 300; case FAILSAFE: return 250; default: return 1000; } }	Adaptive behaviour component. The controller checks sensors more frequently in higher-risk states and less frequently in safe or idle states.
cpp bool irrigationDemandActive() { return soilDry && tempHigh && !waterLow && !waterFailsafe && sensorsValid;	This demonstrates sensor fusion because irrigation demand depends on soil dryness, temperature, water safety, and sensor validity.
if (!sensorsValid waterFailsafe) {if (currentState != FAILSAFE) {changeState(FAILSAFE); } return;}	This protects the system by immediately entering failsafe if a sensor is invalid or water level is critically low.
if (!waterLow && !waterFailsafe && sensorsValid) {digitalWrite(RELAY_PIN, RELAY_ON);} else {digitalWrite(RELAY_PIN, RELAY_OFF);}	Even in the irrigation state, the pump is only allowed to run if water and sensor conditions are safe.

9. Results / Demonstration Summary

The final Wokwi simulation successfully demonstrated the A.I.U operating across all five FSM states. The LCD, RGB LED, buzzer, relay, and Serial Monitor provided clear feedback for each operating mode.

<u>Test</u>	<u>Expected result</u>	<u>Outcome</u>
Startup	System begins in IDLE	Successful
Button press	Enters MONITORING	Successful
Dry soil + high temperature + safe water	Enters IRRIGATING, pump on	Successful
Irrigation timeout	Enters WARNING, pump off	Successful
Low water below 25%	Enters WARNING, pump locked off	Successful
Critical water below 10%	Enters FAILSAFE	Successful
Water recovery above 15% + button	Returns to IDLE	Successful
Soil hysteresis	Dry condition does not flicker near threshold	Successful
Adaptive polling	Serial Monitor shows changing sensor intervals	Successful
Night heat-risk	Lower heat threshold used at night	Successful

Some limitations for the project and challenges encountered were the Wokwi uses simulated sensor values rather than real agricultural hardware. The soil moisture and water level are represented by potentiometers. The pump is represented by a relay module rather than a physical irrigation pump. Also, the threshold values are chosen for demonstration and would need calibration for real-world deployment.

10. Individual Contributions

<u>Member</u>	<u>Contribution</u>
Thomas Dosbon	Designed the A.I.U concept, built the Wokwi circuit, implemented the FSM, sensor processing, adaptive polling, safety logic, output control, testing, report, and demonstration.